

Dictionnaire des données et décisions d'architecture

1. Dictionnaire des termes

Terme	Définition
Utilisateur	Compte permettant d'accéder à l'application et aux fonctionnalités autorisées.
Profil	Rôle associé à un compte : Visiteur, Collectionneur, Acheteur, Vendeur ou Donateur. Un compte peut cumuler plusieurs profils.
Carte	Représentation numérique d'un modèle de voiture disponible sur le marché.
Exemplaire	Unité d'une carte appartenant à un seul compte utilisateur.
Rareté	Niveau de rareté d'une carte déterminé par le nombre d'exemplaires en circulation. Elle n'est jamais calculée localement par l'application.
Cours	Prix officiel actuel d'une carte. Il est calculé et fourni uniquement par le serveur.
Offre de vente	Proposition d'un utilisateur de vendre un exemplaire d'une carte.
Réservation	Immobilisation temporaire d'une carte jusqu'au paiement, à l'annulation ou à l'expiration.
Ordre conditionnel	Ordre d'achat ou de vente déclenché automatiquement lorsque le cours atteint un seuil défini.
Transaction	Opération validée par le serveur permettant notamment l'achat, la vente ou le transfert d'une carte.
Portefeuille	Vue permettant de consulter les cartes possédées, leur valeur et les performances associées.
Notification	Information envoyée à l'utilisateur concernant notamment une transaction, un ordre exécuté ou un cadeau reçu.
Cache local	Données conservées temporairement sur l'appareil afin d'améliorer la disponibilité et la reprise après une perte réseau.
Jeton	Information d'authentification permettant de maintenir la session de l'utilisateur. Il doit être stocké de manière sécurisée.
API REST	Interface permettant à l'application mobile d'échanger des données avec le serveur en HTTPS/JSON.
Temps réel	Mécanisme permettant de recevoir les changements du serveur via SignalR/ WebSocket.
Idempotence	Mécanisme empêchant qu'une même action utilisateur déclenche plusieurs fois une transaction.

Les notions de carte, exemplaire, réservation et ordre conditionnel ainsi que les règles relatives au cours et à l'idempotence sont définies dans le cahier des charges.

2. Dictionnaire des principaux états

Offre de vente

Une offre peut prendre les états suivants :

Brouillon → Active → Réservée → Vendue

Des états alternatifs sont également possibles :

- Active → Retirée
- Active → Expirée

Réservation

Une réservation peut être :

Demandée → Active → Confirmée

Ou prendre un état alternatif :

- Active → Annulée
- Active → Expirée
- Demandée → Rejetée

Ordre conditionnel

Un ordre automatique possède les états :

Actif → En cours → Exécuté

Avec également :

- Actif → Annulé
- Actif → Expiré
- En cours → Rejeté

Connexion temps réel

La connexion SignalR/WebSocket possède les états :

Déconnectée → Connexion → Connectée

En cas de problème :

Connectée → Reconnexion → Connectée

ou :

Reconnexion → Arrêtée

Ces états sont explicitement prévus par le cahier des charges.

3. Décisions d'architecture

DA001 — Utilisation de .NET MAUI

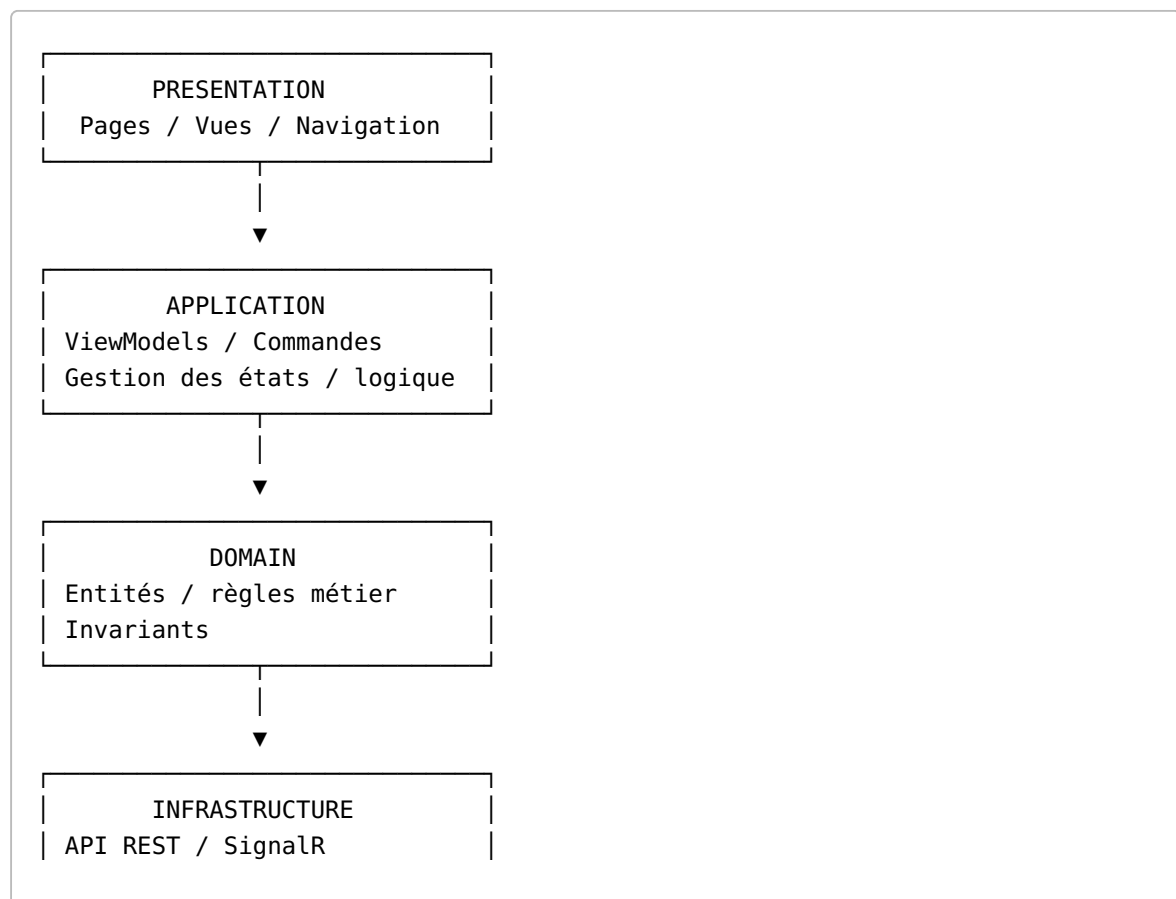
Décision : utiliser .NET MAUI pour développer l'application mobile.

Justification : le projet demande une application mobile .NET MAUI compilable et documentée.

DA002 — Architecture MVVM

Décision : utiliser le modèle **MVVM (Model-View-ViewModel)**.

Organisation



Cette séparation correspond aux couches imposées dans le cahier des charges.

DA003 — Séparation stricte des couches

Décision : une page ne doit jamais communiquer directement avec le client HTTP ou modifier directement le cache.

Les responsabilités sont séparées :

- **Presentation** : affichage et navigation.
- **Application** : commandes, ViewModels et gestion des états.
- **Domain** : entités et règles métier.
- **Infrastructure** : API, temps réel, cache et stockage sécurisé.
- **Tests** : tests unitaires, intégration et mocks.

Cette séparation améliore la maintenabilité et facilite les tests.

DA004 — Serveur comme source de vérité

Décision : le serveur est l'autorité pour les données métier importantes.

L'application mobile :

- ne calcule pas elle-même le cours officiel ;
- ne déduit pas la rareté ;
- ne confirme pas seule une transaction ;
- demande au serveur de valider et d'exécuter les opérations.

Le cahier des charges précise que le serveur est l'unique source du cours officiel et qu'il contrôle et historise les transactions.

DA005 — Communication avec l'API REST

Décision : utiliser une communication **HTTPS + JSON** avec le serveur.

L'accès à l'API est centralisé dans les services de l'application afin d'éviter que chaque écran gère directement les requêtes HTTP.

Le cahier des charges prévoit notamment 19 routes REST et impose une configuration centralisée ainsi qu'un identifiant de corrélation.

DA006 — Utilisation de SignalR/WebSocket

Décision : utiliser SignalR/WebSocket pour les informations nécessitant une mise à jour en temps réel.

Les événements prévus sont notamment :

- `quote.updated` → modification du cours ;
- `offer.updated` → modification d'une offre ;
- `reservation.updated` → modification d'une réservation ;
- `order.executed` → exécution d'un ordre ;
- `gift.received` → réception d'un cadeau.

En cas de coupure, l'application doit marquer les données comme périmées, tenter une reconnexion puis effectuer un rechargement REST après reconnexion.

DA007 — Stockage sécurisé de l'authentification

Décision : stocker le jeton d'authentification uniquement dans un stockage sécurisé.

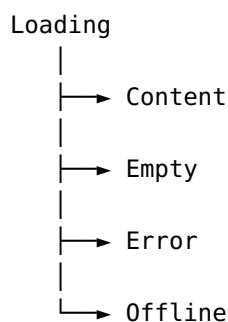
Le jeton ne doit :

- ni être enregistré en clair ;
- ni apparaître dans les logs ;
- ni être placé dans le dépôt Git.

Lors de la déconnexion, le cache associé au compte doit être nettoyé.

DA008 — Gestion des états de l'application

Décision : chaque écran doit gérer explicitement plusieurs états :



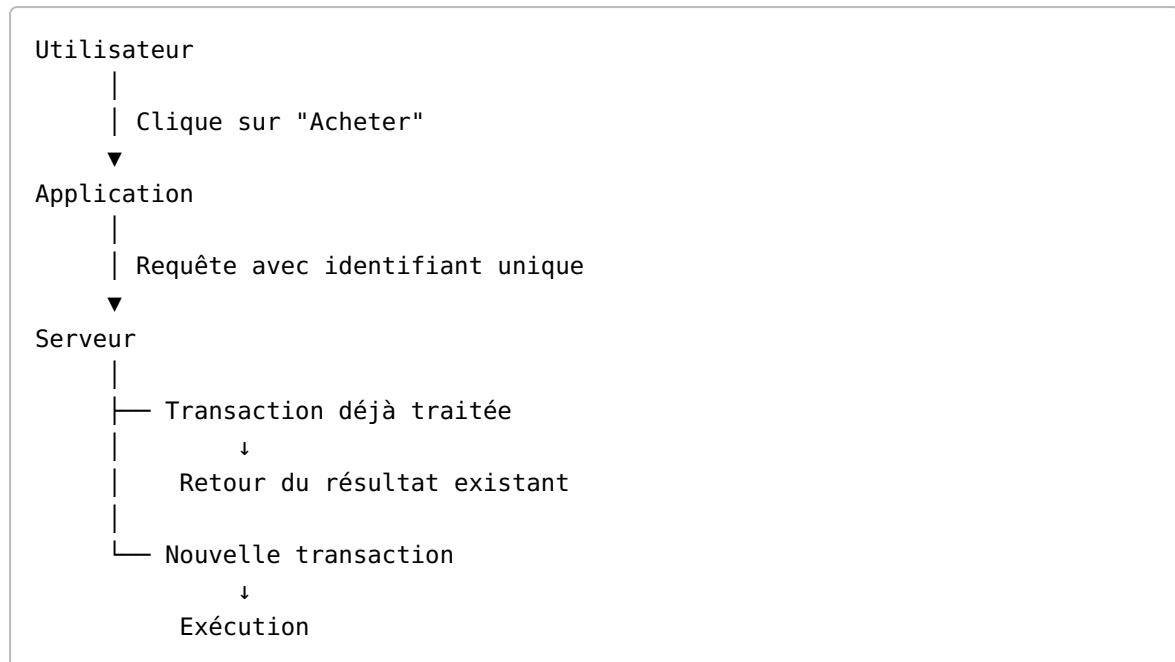
Cela permet d'informer clairement l'utilisateur lorsque les données sont en cours de chargement, absentes, indisponibles ou périmées.

Cette approche est demandée par le cahier des charges.

DA009 — Gestion de l'idempotence

Décision : les actions sensibles comme l'achat doivent être protégées contre les doubles soumissions.

Exemple :



Cela permet d'éviter qu'un double clic ou une répétition de requête crée deux transactions.

L'idempotence est une règle métier obligatoire du projet.

4. Synthèse des choix

Décision	Choix retenu	Objectif
Framework	.NET MAUI	Application mobile
Architecture	MVVM	Séparation des responsabilités
Communication	HTTPS / JSON	API REST sécurisée
Temps réel	SignalR / WebSocket	Mise à jour des cours et événements
Authentification	Jeton sécurisé	Protection de la session
Données métier	Serveur comme source de vérité	Éviter les incohérences
Cache	Cache local isolé par compte	Résilience réseau
Tests	Unitaires + intégration + mocks	Vérification de la qualité

Décision	Choix retenu	Objectif
Transactions	Idempotence	Éviter les doublons
États UI	Loading / Content / Empty / Error / Offline	Meilleure expérience utilisateur

Conclusion

L'architecture retenue repose donc sur une application **.NET MAUI en MVVM**, organisée en couches **Presentation, Application, Domain et Infrastructure**. L'application communique avec le serveur via une **API REST sécurisée** et utilise **SignalR/WebSocket** pour les informations temps réel.

Le serveur reste l'autorité pour les cours, les propriétés, les réservations et les transactions. L'application mobile est principalement responsable de l'interface, de la gestion des interactions utilisateur, de la communication avec les services et de la présentation des données.